

Apuntes SSOO

Víctor Manuel Peiró Martínez

FIIS 2ºA

Sistemas Operativos

Tema 1 - Introducción

¿Que es un Sistema Operativo?

Definición: El programa más importante que se ejecuta en la máquina

Funciones:

- Abstracción de los detalles físicos
 - Ejecuta procesos en entornos virtuales y accede recursos usando las primitivas del sistema
- Permiten que múltiples aplicaciones y usuarios compartan el mismo equipo de forma segura
 - Los programas también son usuarios del sistema operativo

Capas:

- Usuarios y Programas de Usuarios
- Capa de Servicios
 - Llama al sistema
 - Conjunto de instrucciones que van a poder ser llamadas por los programas de usuarios
 - En Unix: API POSIX y en Windows: Win32
- Kernel o Núcleo
 - Gestor de Memoria
 - Gestor de Ficheros
 - Gestor de Procesos
 - Gestor de Seguridad
 - Gestor de Red
 - Gestor de Usuarios
 - Si no se puede realizar un proceso, va a darle prioridad a otro proceso y eso lo interrumpe
- Hardware
 - Subsistemas E/S
 - Drivers
 - Hardware

La Máquina Desnuda

Definición: Una máquina sin sistema operativo

Ejemplo: Un arduino o un reloj digital sencillo

Máquina Vestida: Máquina que contiene SO

La unidad mínima direccionable de memoria es la palabra que consta de un byte.

La máquina se organiza en posiciones de 16 bits.

Las instrucciones se organizan en 16 bits. 12 son la operación y 4 son la dirección implicada

La máquina solo puede ejecutar una instrucción a la vez

Limitaciones:

- Ineficiente gestión de recursos
- Seguridad
 - Los programas del usuario pueden acceder a cualquier recurso del ordenador
- Nula Abstracción
-

Arquitectura Von Neumann

El cerebro del ordenador es la CPU (Unidad Central de Procesamiento)

Consta de:

- ALU (Unidad Aritmético-Lógica)
- Unidad de Control
- Registros de propósito general y especial

Concepto del Programa Almacenado:

- Los programas y los datos se almacenan en memoria
- El CPU funciona según un ciclo de lectura, decodificación y ejecución de cada instrucción
- El contador del programa accede a la siguiente instrucción

Recursos del Ordenador

El SO consigue que cada proceso funcione como si tuviese el equipo entero a su disposición:

- **Scheduling:** Repartiendo el uso de la CPU
- Planificación multicore
- Manteniendo una jerarquía de memoria
- Gestionar acceso al almacenamiento secundario
- Manejo de dispositivos E/S
- Permite la comunicación entre procesos y su sincronización

Modos Usuario/Supervisor

La ejecución simultánea plantea varios problemas de seguridad:

- No queremos que un programa acceda a memoria del otro, maliciosamente o por error
- No queremos que el crash de un programa afecte a todo el sistema
- No queremos que cualquier programa atienda interrupciones
- No queremos corromper dispositivos de entrada/salida o ficheros

Esto se soluciona haciendo que algunas instrucciones de código sólo puedan ser ejecutadas en modo supervisor. Solo el SO tiene permiso para cambiar de modo usuario a supervisor mediante el registro de flags.

Si una aplicación del usuario tiene que acceder a HW, tiene que pasar por el SO obligatoriamente.

Tipos de SO

De propósito general:

- Uso General: Linux, Windows, Mac OS
- Servidores: Unix, Linux

Moviles:

- Android, iOS, Ubuntu Touch

Empotrados:

- RTOS, Windows IoT, Raspbian, Contiki

De tiempo real:

- QNX, VxWorks, RTLinux

Historia de los SO

Comenzó con los ordenadores de segunda generación, el operador leía los datos y el programa desde tarjetas perforadas. Se graba en cintas y se carga y se ejecuta en el mainframe.

Los **sistemas de procesamiento por lotes** realizan estos trabajos automáticamente y generan una cinta de ejecución secuencial.

Se desarrollan los primeros SO modernos en los años 60, con multiprogramación y multiusuario

Multiplexaje

Multiplexaje: Gestión de los recursos en función del tiempo y espacio

Cuando un recurso se multiplexa en el tiempo:

- Distintos programas toman turnos para utilizar el SO
- Se ejecuta uno, cuando se ha ejecutado el tiempo suficiente, se da acceso a otro y luego se vuelve el primero hasta el fin de la ejecución

Cuando un recurso se ejecuta en el espacio:

- Cada usuario obtiene una parte del recurso.
- La memoria principal y los discos duros se dividen en varios programas de ejecución.

Multics

Desarrolló muchas partes del SO pero fue un fracaso comercial ya que estaba muy adelantado a su tiempo.

Implementa:

- Memoria Virtual y Segmentada
- Jerarquías de Ficheros
- Bases de Datos Relacionales
- Multicore y memoria compartida
- Seguridad

Tema 2 - Conceptos Basicos

Conceptos

El SO tiene 2 misiones fundamentales:

- Gestionar los recursos de manera que los programas vean una máquina completa a su disposición
- Abstractar los detalles de los dispositivos hardware

El Sistema Operativo gestiona y crea procesos.

Proceso: Instancia ejecutable de un programa

Cada proceso requiere código, datos y metadatos

Dos instancias de un ejecutable son procesos distintos

El sistema ofrece una API normalizada hacia los programas para poder invocar sus servicios

Kernel:

- Parte más importante del sistema operativo
- Se ejecuta en un modo privilegiado con acceso a toda la memoria, recursos y registros de la CPU.

Procesos

Stack:

- El código ejecutable, datos y funciones se llama aquí
- Los parámetros y variables se instancian aquí

Heap:

- Se usa para el manejo de memoria dinámica.

Tanto el stack como el heap son de tamaño variable

La unidad mínima de ejecución es el código máquina.

Cuando el CU comienza la decodificación, esta debe finalizar. Esto causa que solo se puede estar un proceso en ejecución en cada instante por cada núcleo

Como los procesos no siempre están activos, el SO puede ir concediendo la CPU por turnos a los procesos dando la sensación de que se procesan en paralelo.

Cada vez que hay una interrupción, el proceso es desalojado de la CPU.

El ciclo de vida de un proceso se representa de la siguiente manera:



Arranque del Sistema Operativo

1. Reset

- El microprocesador recibe una señal por el pin RESET cuando se enciende el equipo. Esto le da un valor inicial a todos los registros.

- Así se consigue que se empiece a ejecutar código máquina en una dirección especial con un salto de rutina al siguiente paso.

2. Iniciador Hardware

- El equipo salta a un programa en el ROM o flash proporcionado por el fabricante (BIOS/UEFI)
- Se procede a detectar los dispositivos conectados y hacer una prueba mínima de funcionamiento. Esto se llama POST (Power-On Self-Test).
- El firmware busca un dispositivo de almacenamiento masivo que tenga un sector de arranque configurado siguiendo las instrucciones del BIOS.
- Se carga en memoria el contenido del MBR (Master-Boot Register) de la unidad de arranque y se cede el control
- El MBR ocupa el primer sector y contiene 512 bytes de código donde puede haber saltos hacia el cargador del sistema o un completo selector de arranque
- Este código no forma parte del SO y es autosuficiente.

3. Carga del Sistema Operativo

- Se empieza a cargar en memoria el SO residente, la parte residente (kernel y algún driver).
- Se crean las tablas de gestión y cuando concluye con éxito se habilitan las interrupciones y se crea el proceso init/systemd en Unix.
- Init lee los scripts almacenados en /etc/init para ir creando el resto de procesos que arrancan automáticamente ejecutándose en modo usuario

Activación del Sistema Operativo

El kernel está siempre almacenado en la RAM para evitar el coste de leer su código cada vez. La única manera de despertar al kernel es mediante una interrupción.

Cuando ocurre esta interrupción, el CPU lo sabe porque recibe una señal por el pin INTR o NMI.

Al procesar la interrupción, hay que poner el PC a la dirección del código de atención pero perdemos el contenido que apuntaba a la siguiente instrucción. Esto se soluciona guardando el estado del proceso en la pila.

La CPU termina de ejecutar la instrucción del código máquina en curso, salva el PC y el registro de flags y se carga en el PC la dirección que contiene la rutina de atención a las interrupciones poniendo al CPU en modo supervisor.

- La rutina salva el estado en el stack del usuario y reconoce el dispositivo que requiere atención.
- Con esto se accede a el código concreto localizado en la tabla de vectores de interrupción
- Cuando finaliza la atención, se recuperan los valores y se vuelve al modo usuario.

Tabla de Vectores de Interrupción del 8086:

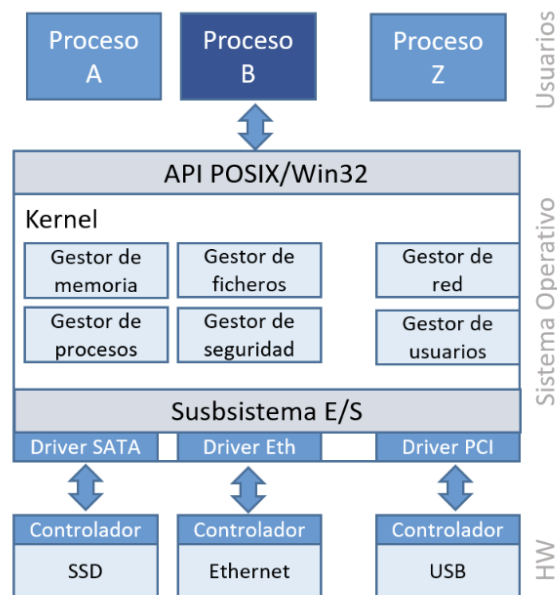
- Los valores más bajos son para atender excepciones e interrupciones SW.
- A continuación, hay un rango de interrupciones predefinidas para la atención de los circuitos.
- Quedan 224 libres para manejar otros dispositivos de entrada y salida.

Tipos de Interrupciones:

- **Hardware:**
 - **Tick:** Señal eléctrica general a partir del reloj de la CPU mediante un divisor de frecuencia.
 - Garantiza que el kernel se despierte a menudo para poder llevar a menudo la función de planificación
 - **Función de Planificación:** Decide si el proceso actual debe ser destituido.
- **Software:** Una instrucción de código máquina pone el CPU en modo interrupción y tiene una entrada en la tabla de vectores
 - Se invoca colocando en el stack el código de la operación que queremos que realice el sistema, despertarlos con interrupción SW y recibir el valor de vuelta

POSIX: Colección de estándares de IEEE que definen una API que debe ofrecer el sistema.

- En Linux es Unix y en Windows en el Win32/Win64



Gestión de Memoria

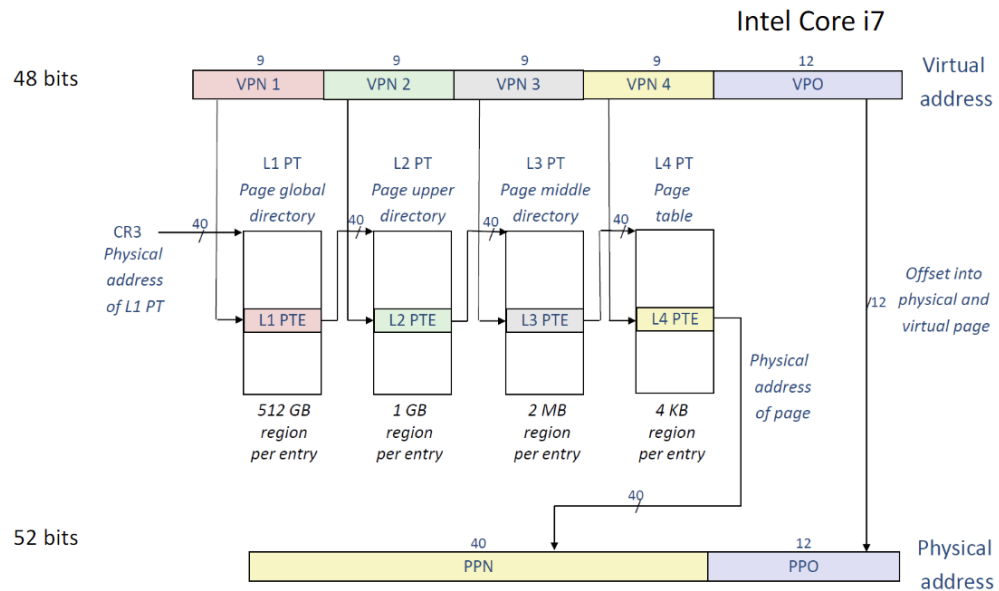
Gestionar un SO no sería posible sin un mapa de memoria cargado en la RAM

Una combinación de circuitos de la CPU permite tener cargados solo los fragmentos del código en ejecución a corto plazo mientras que el resto se está leyendo desde o escribiendo a un disco cuando no se usa.

Esta gestión la realiza el kernel con memoria virtual con paginación.

El SO oculta esta complicada gestión y ofrece memoria dinámica y creación de segmentos de memoria compartida para la comunicación de procesos.

El equipo supervisa por hardware que ningún proceso accede a direcciones indebidas.
 Tabla de memoria de i7: Traduce 48 direcciones virtuales en 52 físicas.



Comunicación y Sincronización

Mecanismos:

- Señales
- Pipes
- Colas de mensajería
- Sockets
- Memoria Compartida

Todos estos servicios se manejan desde el kernel

Cuando varios procesos intentan acceder a un mismo recurso pueden surgir condiciones de carrera.

La única manera de evitar esto es proporcionando un mecanismo que asegure la ejecución de un fragmento sin que el sistema sea expulsado por otro que quiera acceso.

El sistema ofrece semáforos y mutexes como solución

Gestión de Ficheros

Los sistemas necesitan un mecanismo de almacenamiento secundario no volátil para guardar datos.

Fichero: Conjunto de información organizada accesible de forma secuencial y con unos metadatos asociados

Gran flexibilidad ha permitido transplantarla a tecnologías como los SSD, memorias SD y los CD-ROM.

El gestor de ficheros del SO oculta los detalles particulares de los dispositivos de almacenamiento y presenta la información con la abstracción fichero.

Gestión de Entrada/Salida

Cada equipo ofrece una interfaz no estándar que presenta unos registros en el circuito propio llamado controlador.

La sintaxis y semántica de la comunicación con los dispositivos E/S la abstrae un driver. El driver es un módulo especial que se enlaza directamente con el kernel y permite la comunicación con un dispositivo específico

La abstracción de dispositivos es muy similar a la de ficheros ya que la mayoría pueden tratarse como ficheros

Arquitectura x86

Nombre	Bits Arquitectura	Bits Bus Direcciones	RAM
4004	4	4	64 bits
8008	8	8	$2^8 = 256$ bits
8080	8	16	$2^{16} = 64$ kB
8086	16	20	$2^{20} = 1$ MB
80286	16	24	$2^{24} = 16$ MB
80386	32	32	4GB

Modelo 8086

Arquitectura de 16 bits con bus de direcciones de 20 bits(1 MB de RAM)

No tiene modo supervisor

Registros:

- Cada registros tiene su propia seguridad
- AX a DX → Registros de propósito general. AX es el acumulador
 - Se dividen en AH y AL. Subregistros de 8 bits
- SP: Stack Pointer
- BP: Base Pointer
- DI,SI: Registros de Indirección
- IP: Instruction pointer. Contador de Programa

- CS,DS,SS,ES: Registros de Segmento

Modelo 80386

Arquitectura de 32 bits con bus de direcciones de 32 bits(4GB de RAM)

Registros:

- EAX a EDX: Se expanden a 32bits
 - Se permite acceso a 16 bits(AX) y a los de 8 bits soportados por el 8086
- Los registros de manejo también se expanden a 32 bits
- 3 registros especiales: CR0 a CR3:
 - Supervisor fijada con un bit en CR0

Con un bus de direcciones de 48 bits aparecen nuevos registros:

- RAX a RDX:
 - Registros expandidos a 64 con compatibilidad a 32,16 y 8 bits
- R8 a R15:
 - Nuevos registros de 16 bits
- Se amplían los registros de manejo

Manejo de Stack

El registro SP apunta al último elemento añadido.

Cuando se invoca a una función, el stack almacena:

- Los parámetros de entrada
- La dirección de entorno
- Las variables locales de la función

ARM

Advanced RISC Machine

RISC: Conjunto reducido de instrucciones

Micros de 64 bits

Más simples que CISC como x86

Son más eficientes energéticamente y el diseño hardware es más fácil.

Los compiladores son más complejos

Juego de Instrucciones de tipo Load and Store

No puede hacer operaciones directamente sobre memoria solo sobre registros

Permite manejar memoria virtual pero en móviles no usa swap

Virtualización

Hipervisor: Recurso de permite la virtualización de varias máquinas virtuales en una máquina física

Demonio: Proceso que se ejecuta en segundo plano

Contenedor: Paquete de software que contiene todo el entorno necesario para su ejecución

- Se ejecutan en un servidor que se monta sobre el SO anfitrión

Tipo 1:

- El hipervisor tiene acceso a los recurso hardware y se ejecutan en el

Tipo 2:

- El hipervisor se consigue mediante un aplicación software en el sistema operativo

Tecnología Docker:

- Virtualización más ligera
- Diseñada para una sola aplicación
- Funciona como un modelo cliente-servidor. El cliente puede ser una shell de comandos o un API.
- No tiene porque estar en la misma máquina que el demonio
- El usuario se pone en contacto con el docker daemon.
- Registry es el repositorio de imagen
 - Las imagenes son solo de lectura así que no se pueden hacer cambios a ella

Tema 3 - Gestión de Procesos

Proceso

Un proceso es un programa en ejecución

Un proceso consta de una imagen de memoria, compuesta por código ejecutable, sus variables, el heap, el stack y las librerías que usa si son dinámicas

La imagen se instancia en la memoria física del ordenador de forma completa

La imagen tiene su stack y el kernel tiene el suyo propio

Mapa de Proceso

El proceso necesita varias informaciones para funcionar:

- **Registros de CPU:**
 - Los registros de propósito general contienen datos de la su propiedad
 - También es información crítica la información del PC, IR y SP
 - Si el proceso se expulsa de la CPU, los registros se guardan en el BCP para poder volver al punto correcto de la ejecución
- **Entorno:**
 - Datos globales que hereda el proceso a nacer como \$PATH y &HOME
- **Metadatos:**
 - Datos sobre el proceso durante la ejecución y residen en la memoria del kernel
 - EL BCP(Bloque de Control de Procesos) guarda las identidad del proceso, su prioridad, su estado, etc..
 - La tabla de páginas guarda la correspondencia entre la memoria virtual y los marcos en los que está cargado el proceso
 - El mapa de E/S almacena información sobre los descriptores de E/S y ficheros que se usan
- **Disco:**
 - Se guarda el código ejecutable del programa
 - Se guarda las páginas de la zona swap si hay memoria virtual

En xv6

La imagen se instancia desde 00000000H y 80000000H

Por encima se ubica el kernel hasta el FE000000H

Dos stacks. Del usuario y del kernel

La existencia de 2 pilas permite no tener que hacer cambio de contexto solo por el paso de usuario a supervisor

Aunque la pila del proceso se corrompa, la del kernel no sufre y puede tratar la excepción

Cada proceso tiene su BCP

En Linux

En linux se llama task_struct

Se instancia en la pila del espacio del kernel del proceso

Los BCP se organizan como una lista circular doblemente encadenada

- Cada task_struct tiene un puntero hacia el stack del proceso anterior y siguiente

Cuando se crea un proceso, el SO tiene que se capaz de insertar o eliminar el BCP de la lista

Ciclo de Vida del Proceso

La vida en de un proceso abarca desde su invocación a su liberación

Creación

El proceso solo se crea por proceso del otro

Cuando invocamos cualquier comando, la shell es el proceso padre de ese comando

Operaciones necesarias:

- Asignar la imagen de memoria. Si se usa memoria virtual, crea y rellena la tabla de páginas
- Seleccionar un BCP libre en la tabla de páginas del kernel y rellenarla con datos
- Cargar el ejecutable y los datos estáticos inicializados
- Crear el stack y el heap

Hasta que se complete esto, el proceso está en estado nuevo. Cuando se complete, estará en estado listo

Activación

Poner el proceso en ejecución. Se ocupa el kernel con un dispatcher.

Al activar un proceso, hay que recuperar los valores en los registros del micro antes de la última interrupción, almacenados en el BCP.

El kernel entra en modo privilegiado, salva el contexto del modo proceso que estaba en la CPU y rescata el nuevo del BCP del proceso seleccionado y antes de volver al modo usuario, coloca en el PC en valor de retorno de la dirección siguiente a aquella en la que se interrumpió el proceso activado.

Interrupción

La interrupción activa el kernel y hace que el procesador se ejecute en modo privilegiado

Primero, el kernel atiende la interrupción y después salva los registros del procesador visibles por el usuario en el stack del usuario que ejecuta el proceso.

El kernel puede haber bloqueado al proceso como consecuencia de la interrupción o haberlo expulsado de la cola de procesos en espera. Si esto ocurre, el scheduler decide que proceso entra en el CPU y hace un cambio de contexto.

Si el proceso actual puede seguir ejecutando, el kernel recupera el contexto y regresa al entorno del usuario.

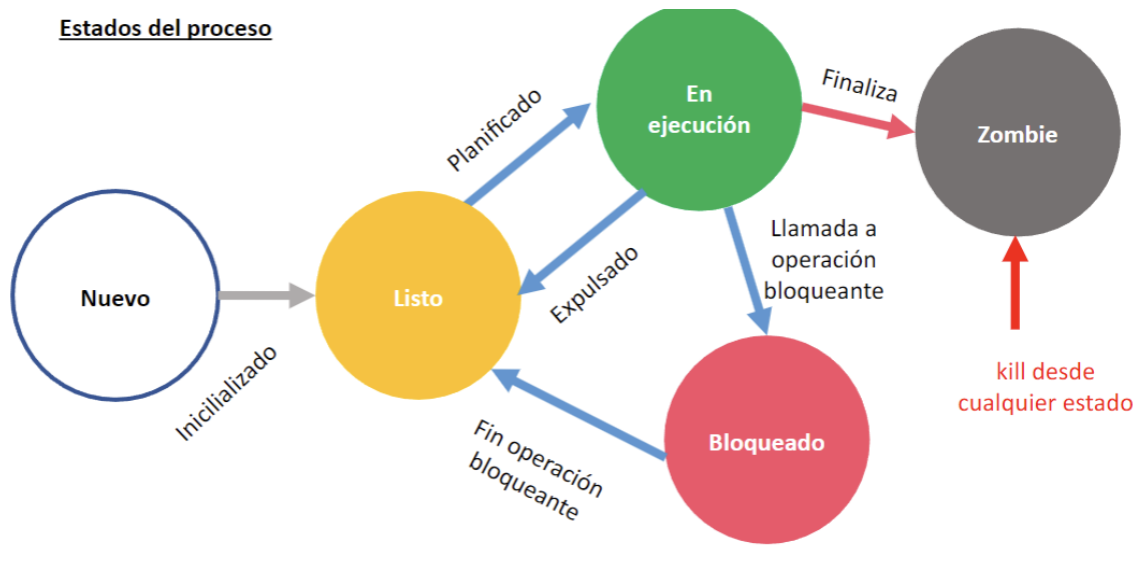
Finalización

Un proceso termina cuando un usuario lo decide.

También puede finalizar debido a una situación anormal como una división entre 0.

Cuando se finaliza, hay que liberar su BCP, tabla de memoria y funciones que dependen del proceso muerto como wait()).

Estados del proceso



Creación

Todos los procesos descienden de un único padre llamado init.

Se pueden crear procesos de 2 maneras:

- **fork()**: Crea un clon del proceso que lo invoca.
 - Si mayor que 0: Proceso Padre
 - Si igual a 0: Proceso Hijo
- **exec()**:
 - Crea un proceso con un ejecutable distinto

Cambio de Contexto

Cambio de Contexto: Procedimiento de desalojar un proceso de la CPU y sustituirlo por otro. El scheduler toma una decisión y el dispatcher se encarga de ello.

Cuando ocurre, el estado del procesador se guarda en el stack del usuario, se atiende una interrupción y se recupera el estado.

Un cambio de contexto puede ocurrir como consecuencia de una interrupción, pero no siempre.

No hay cambio en la de la tabla de páginas de memoria ni del código ejecutable.

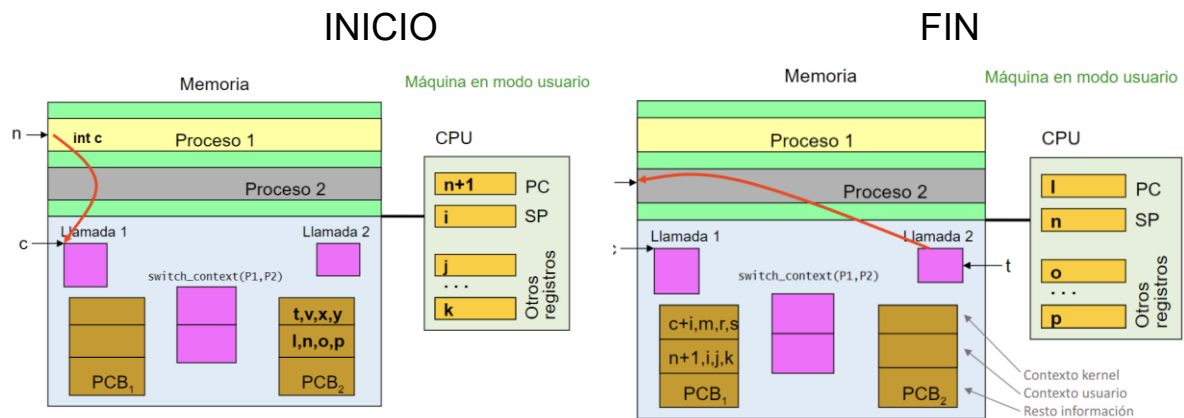
En un cambio de contexto, hay que salvar los registros del procesador cuando se produce la interrupción en su BCP y guardar el estado de los registros cuando el kernel llama al cambio de contexto. Se recuperan los valores de los registros del BCP del que se va a instanciar y se le concede el control.

xv6 crea una estructura trapframe que almacena el estado del procesador.

Pasos

1. Se produce un interrupción por invocación de un operación bloqueante.
2. La máquina pasa a modo kernel, el CPU salva el contexto del proceso n y empieza a ejecutar código kernel
3. Kernel invoca el switch context

- switch context guarda el contexto del kernel en el PCB_1
- El kernel restaura el contexto del procesador desde PCB_2 con los datos del stack del kernel
- El kernel termina en el entorno de P2 y restaura el estado del procesador con los datos del usuario de P2.
- P2 recupera la ejecución y vuelve a modo usuario



Interrupción

Las interrupciones tienen origen hardware y software.

Toda interrupción hace que el microprocesador salte a una dirección especial donde está una única rutina de atención que forma parte del kernel.

Esta rutina salva el contexto del proceso en el stack del usuario, reconoce el origen por un número que envía por HW y selecciona en una tabla de memoria el código para tratar esa interrupción. Al final restaura el contexto.

Las interrupciones se pueden anidar y enmascarar. También, se dividen en 2 partes.

- **Top Half:** El kernel enmascara todas las interrupciones y reconoce la actual. No se puede interrumpir
- **Bottom Half:** Se programa un tasklet. Es interrumpible
 - **Tasklet:** Forma especial de código que se encola en el kernel para su ejecución posterior

En Intel x86

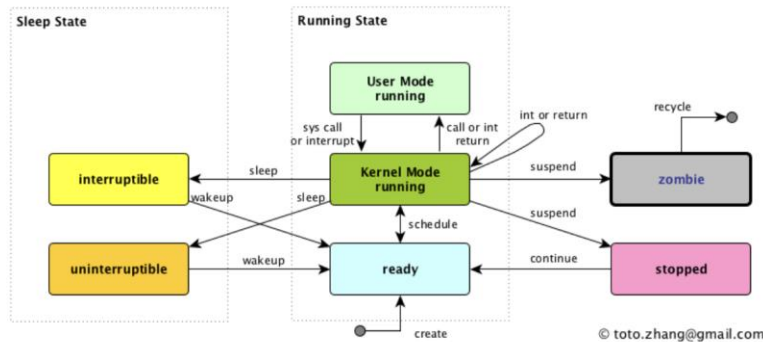
- Dispositivo HW genera interrupción
- Circuito PIC8259A recibe las peticiones y arbitra las mas prioritarias
- Se salva el contexto, reconoce la interrupción y se obtiene el vector de interrupción
- Se termina la rutina de interrupción y se envia señal EOI al 8259 para restaurar el contexto

En Linux

- Kernel reconoce interrupción
- Salva el contexto en stack de usuario
- Salta a rutina de atención

4. Se hace el top half de la atención
5. Se restaura el contexto y se retorna la interrupción
6. Bottom half de la atención

Finalización



Wake Up: Proceso a ready

Procesos Especiales

Proceso Servidor: Proceso que está en bucle infinito pendiente de recibir peticiones del usuario y atenderlas de forma atómica.

Demonio: Proceso que se arranca en el inicio del sistema y es inmortal(si se tumba de forma inesperada vuelve a ejecutarse). Pueden ser servidores y se ejecutan en segundo plano(background) ya que no tienen terminal usuario.

Proceso del Sistema: Proceso que se ejecuta en modo privilegiado.

- Se crean con un servicio especial en el arranque
- Tiene acceso a todo el mapa de la memoria
- Flujos de ejecución independiente
- Solo puede ejecutar procesos en modo usuario

Proceso Nulo: Proceso que no hace nada

Threads

Thread(Hilo): Estructura de datos más compleja nacida para aprovechar las arquitecturas multiprocesador.

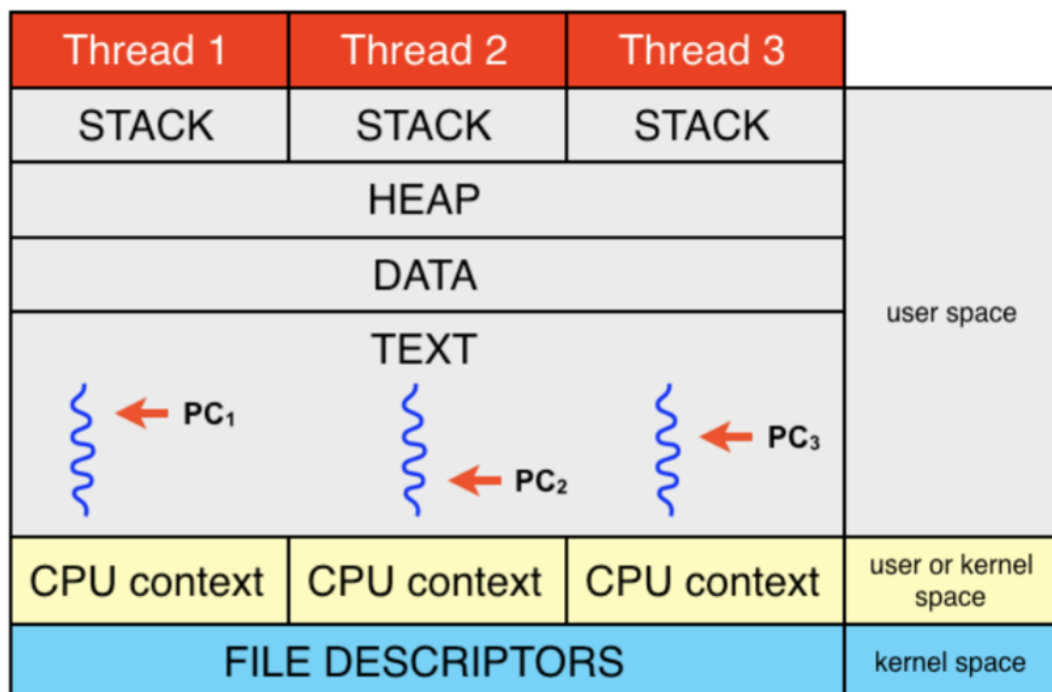
Monothread: Proceso convencional. Su valor está determinado por el valor del PC

Los threads de un mismo proceso comparten código, datos, recursos del SO y tienen su propio PC, registros y stack.

Al acceder a los mismos datos pueden haber conflictos pero aparecen nuevas posibilidades de programación concurrente.

La ventaja de la programación multithread sobre la multiprocesos es que el cambio de contexto es más rápido ya que solo hay que salvar el contexto del thread en una tabla similar a la del BCP de la que es responsable el kernel.

Si un procesador es multicore, cada core/núcleo puede procesar un hilo del mismo proceso. La planificación de threads es la base de los schedulers. Cada hilo tiene su TCB que es una versión reducida del BCP.



Planificación

Planificación del Procesador: Decisión de a qué proceso se le concede la CPU y cuando debe expulsarse

Tipos:

- **Largo Plazo:** Sirve para decidir en qué orden se ejecutan las tareas en un gran sistema en función de las características estadísticas de sus tiempos de ejecución
- **Medio Plazo:** Sirve para decidir qué procesos van a bloquearse
- **Corto Plazo:** Decide que proceso está preparado para la ejecución

Scheduling se refiere a la planificación a medio plazo

Dispatching: Procedimiento de cargar un nuevo proceso en la CPU

Estados del Proceso:

- **Nuevo:** Estado desde que se inicia su carga hasta que ha terminado de ejecutarse por primera vez
- **Listo:** Proceso preparado para ejecutarse
- **En Ejecución:** Único proceso que ocupa la CPU
- **Bloqueado:** El proceso está en cola con otros procesos durmientes
- **Zombie:** Cuando un proceso se elimina, no se borra de inmediato. Estado sin retorno

KPIs: Indicadores de rendimiento que pueden entrar en conflicto

- **Tiempo de Ejecución(t):** Tiempo que necesita un proceso para completarse con el 100% de la CPU
- **Tiempo de Espera/Proceso(E):** Tiempo que está un proceso listo sin ejecutarse
- **Tiempo de Respuesta (T):** Tiempo que está un proceso listo hasta que responde un usuario
- **Penalización(P):** Cociente entre el tiempo de respuesta y el de ejecución. Cuanto más cerca de 1 mejor.

Tick: Interrupción periódica. Se considera la unidad más pequeña de tiempo.

Quantum: Conjunto de ticks. Tiempo máximo que se puede ejecutar un proceso de forma continua habiendo otros en cola

- Garantiza que un único proceso no monopoliza la CPU

En sistemas no expulsivos, el proceso completa su quantum con independencia de los que estén esperando en la cola.

Si expulsivo, el proceso puede ser expulsado aunque no termine su quantum

Algoritmo FCFS

First Come First Serve

El scheduler concede la CPU al proceso que lleve más tiempo esperando en la cola en estado listo.

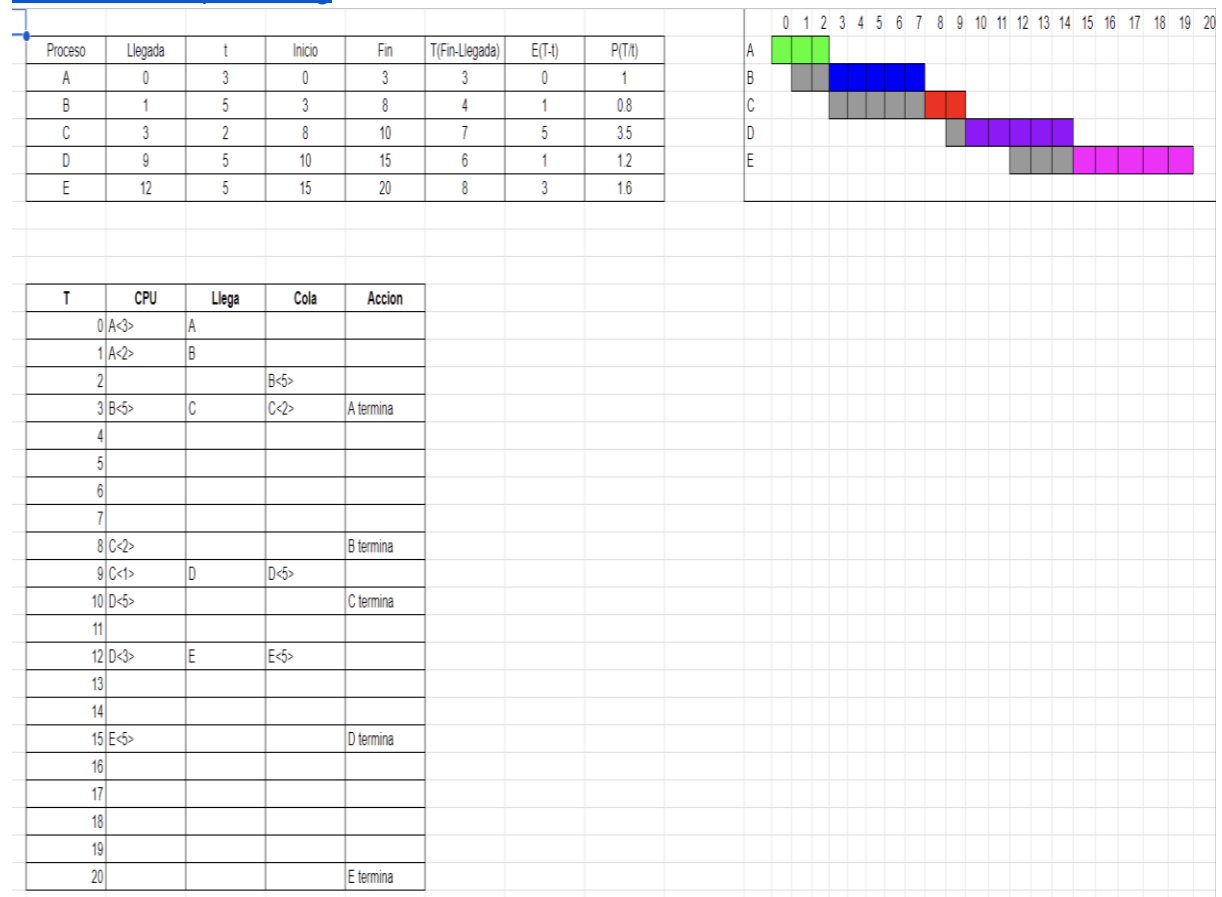
Pasos Resolución

1. Ordenar los procesos según orden de llegada
2. Rellenamos el tiempo de ejecución del proceso
3. Determinamos el tiempo de inicio, para el primero 0 y para el resto coincide con el fin del anterior
4. Se determina el tiempo final
5. Se determina el tiempo de respuesta
6. Se determina el tiempo de espera
7. Se determina la penalización

8. Se determina respuesta (t/T)
9. Se determina el tiempo entre finalización y el instante de llegada. Es igual al de espera

Ejemplo:

https://docs.google.com/spreadsheets/d/1XwJg6VBOIXiK2VoHmZcWB8P_TkgVqcosUv_EidYxPn8/edit?usp=sharing



Round Robin

Mejora sobre el FCFS puro

Un proceso solo puede estar ejecutándose un quantum. Número de ticks predefinido

Si se llega a este límite, el CPU lo expulsa y lo pone en la cola de procesos y concede al proceso con mayor tiempo de espera.

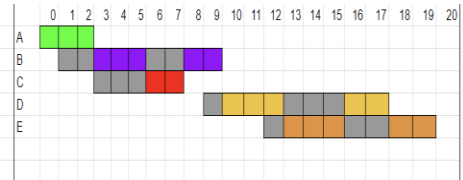
Es importante el tamaño del quantum ya que si es muy grande, el RR se parecerá al FCFS

Ejemplo:

https://docs.google.com/spreadsheets/d/188WfrWsW1UGL0LPGVfME7_XGsJTXqbuyAjMZi7G4x8U/edit?usp=Ejemplo Round Robin=sharing

Quantum 3

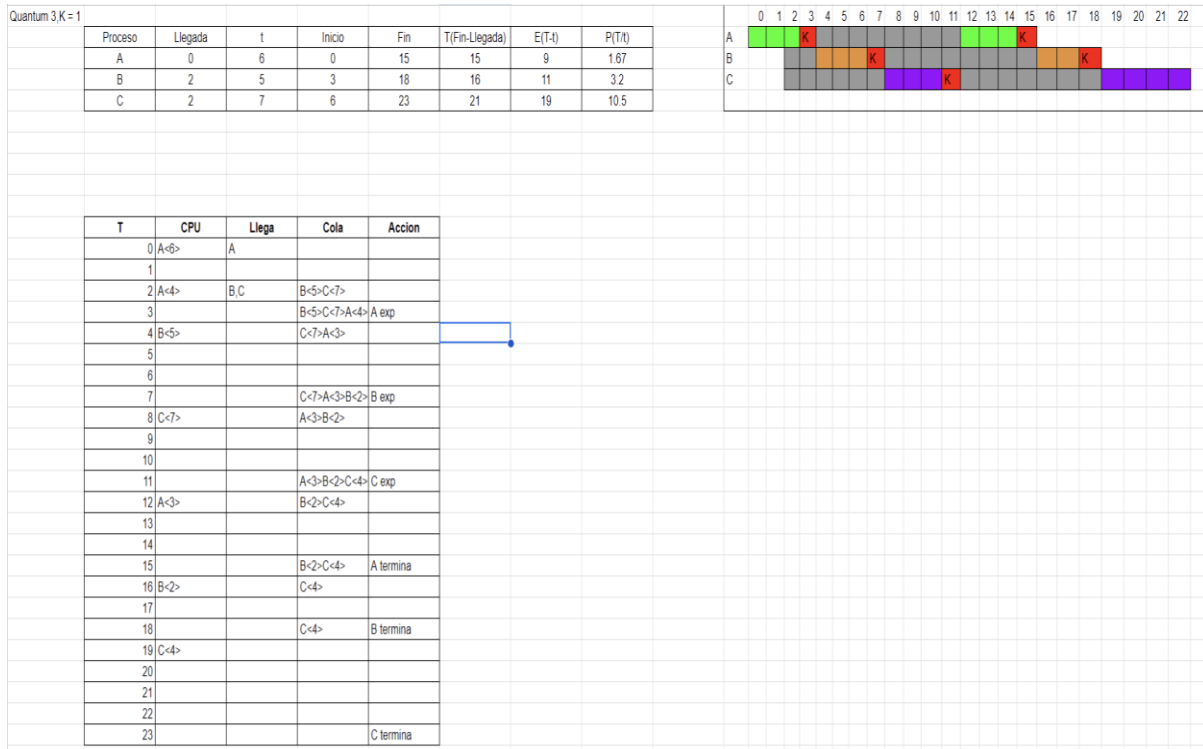
Proceso	Llegada	t	Inicio	Fin	T(Fin-Llegada)	E(T-t)	P(T/t)
A	0	3	0	3	3	0	1
B	1	5	3	10	9	4	1.8
C	3	2	6	8	5	3	2.5
D	9	5	10	17	8	3	1.6
E	12	5	13	20	8	3	1.6



T	CPU	Llega	Cola	Accion
0	A<3>	A		
1	A<2>	B	B<5>	
2				
3	B<5>	C	C<2>	A termina
4				
5				
6	C<2>		B<2>	exp C
7				
8	B<2>			C termina
9	B<1>	D	D<5>	
10	D<5>			B termina
11				
12	D<3>	E	E<5>	
13	E<5>		D<2>	D exp
14				
15				
16	D<2>		E<2>	E exp
17				
18	E<2>			D finaliza
19				
20				E termina

Ejemplo 2: RR con Coste

https://docs.google.com/spreadsheets/d/1Mh0XqkH5d_KKjKxI7SA9p-DGk5-a3a93U8tghA3Kzpw/edit?usp=sharing



Planificación Pt 2

FCFS y RR supone que todos los elementos tienen la misma prioridad pero esto no ocurre. Windows y Linux son capaces de manejar colas de prioridades ya que hay elementos en estado crítico que deben de ser atendidos.

En vez de una sola cola, hay varias colas de prioridad y no se atiende ningún proceso de prioridad reducida hasta que se han atendido todos los de prioridad superior.

La gestión de prioridades conlleva el riesgo de starvation.

- **Starvation:** La CPU es consumida por procesos de alta prioridad y el resto no entran casi nunca

Por eso, las prioridades son dinámicas. Cuando se hayan consumido n quantums, el procesador bajará la prioridad de ese proceso.

También se puede incrementar la prioridad si lleva mucho tiempo en cola

Las colas de prioridad se gestionan con FCFS o RR.

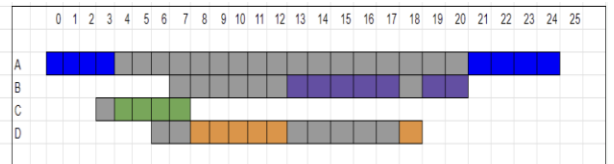
Los quantums de elementos con mayor prioridad son mayores

Prioridad Expulsiva: Se quita la CPU al proceso en curso cuando haya uno más prioritario

Ejemplo: RR con prioridad

https://docs.google.com/spreadsheets/d/1_qqtR6M1djZcasfZqc71rAvhBcE_Y9NhAHoDyLckjLk/edit?usp=sharing

Quantum	Prioridad	Proceso	Llegada	t	Inicio	Fin	T(Fin-Llegada)	E(T-t)	P(T/t)
4	2	A	0	8	0	25	25	17	$25/8 = 3.125$
4	2	C	3	4	4	8	5	1	$5/4 = 1.25$
5	1	D	6	6	8	19	13	7	$13/6 = 2.16$
5	1	B	7	7	13	21	14	7	$14/7 = 2$



T	CPU	Llega	Cola	Accion
0	A<8>	A		
1				
2				
3	A<5>	C	C<4>	
4	C<4>		A<4>	A exp
5				
6	C<2>	D	D<6>A<4>	
7	C<1>	B	D<6>B<7>A<4>	
8	D<6>		B<7>A<4>	C termina
9				
10				
11				
12				
13	B<7>		D<1>A<4>	D exp
14				
15				
16				
17				
18	D<1>		B<2>A<4>	B exp
19	B<2>		A<4>	D termina
20				
21	A<4>			B termina
22				
23				
24				
25				A termina

Proceso	Inicio	Fin	Retardo	Turnaround	Throughput
A	0	25	17	25	1
B	7	21	14	14	1
C	3	8	5	5	1
D	6	19	13	13	1

Herramienta Recortes

Captura de pantalla copiada en
guardada

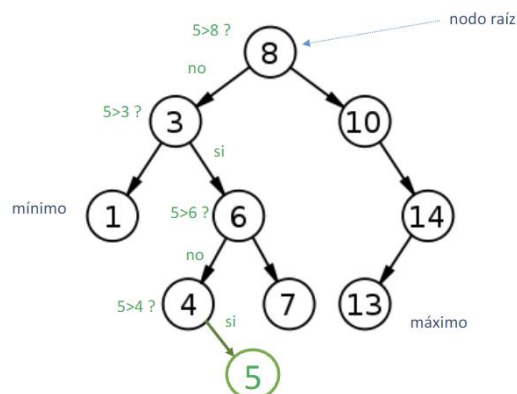
Linux CFS y O(1)

O(1):

- Proceso de Tiempo Real: Prioridades de 0 a 99
- Procesos Normales: Prioridades de 100 a 139
- Prioridad base = 120. Cambiable con nice
- Prioridades en orden descendente

CFS:

- Completely Fair Scheduler
- Usa arboles binarios



Tema 4 - Gestión de Memoria

Jerarquía De Memoria

De tamaño y capacidad pequeñas:

1. Registros del CPU → Muy rápidas y caras
2. Caché de la CPU → Muy rápidas y caras

Capacidad y tamaño medio

3. Memoria RAM → Rapido y economico
4. Memoria Flash/USB → Barato y lento

Tamaño y Capacidad Altas

5. Discos Duros → Lentos y muy baratos
6. Backups de cinta → Muy lentas y económicos

El funcionamiento de esta jerarquía se basa en 2 cosas:

- **Proximidad/Localidad Espacial:** Los programas se ejecutan de forma lineal haciendo que las instrucciones sean consecutivas y los datos residen en zonas acotadas.
- **Proximidad/Localidad Temporal:** Ocurre por la repetición de secuencias de código llamadas funciones

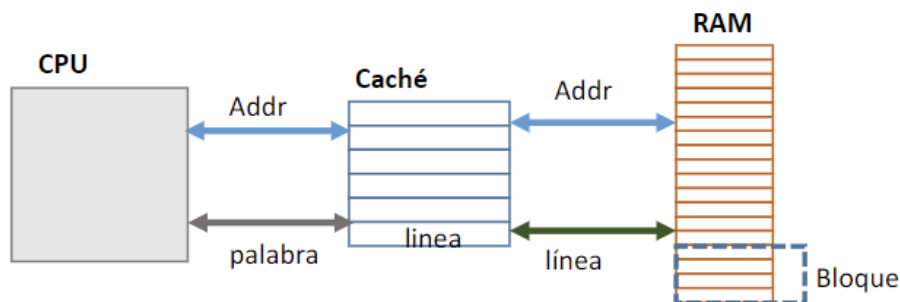
Conjunto Breve: Fragmento de código y datos que se acceden de forma repetida

Nivel de memoria	Capacidad	Tiempo de acceso	Tipo de acceso
Registros	64 a 1024 bytes	0,25 a 0,5 ns	Palabra
cache de memoria principal	8 KiB a 8 MiB	0,5 a 20 ns	Palabra
Memoria principal	128 MiB a 64 GiB	60 a 200 ns	Palabra
Disco electrónico SSD	128 GiB a 1 TiB	50 μ s (lectura)	Sector
Disco magnéticos	256 GiB a 4 TiB	5 a 30 ms	Sector

Memoria Caché

Memoria Caché: Memoria muy pequeña y muy rápida situada cerca del CPU capturando señales de los buses.

Se conecta también con la RAM para intercambiar bloques que coinciden con las líneas en las que se organiza el RAM



El acceso a la RAM es muy lento en comparación con la velocidad del procesador.
Cuando un dato está en la caché, se registra un acierto(hit). Sino, se registra un fallo(miss).

$$t_{ef} = t_{hit} * P(hit) + t_{miss} * P(miss) = t_{hit} * P(hit) + t_{miss} * (1 - P(hit))$$

Ejemplo:

Supongamos que tenemos una caché con un tiempo de acceso de 2ns y una RAM con tiempo de acceso de 50ns. Si la tasa de acierto es de un 90%, ¿cuál es el tiempo efectivo de acceso a memoria?

$$t_{ef} = 2 \times 0.9 + 50 \times 0.1 = \underline{6.8 \text{ ns}}$$

La realidad es más compleja ya que la transferencia entre caché y RAM es en bloques. Aunque aumente el tiempo, el acceso a la caché sigue siendo inferior al tiempo de acceso a la RAM.

El caché se organiza en conjuntos de palabras(líneas) que se transfieren desde o hacia la RAM.

El tamaño de la línea en caché es igual al tamaño del bloque en la RAM

Cada entrada en la caché tiene un campo llamado etiqueta

Ejemplo:

Una CPU tiene bus de direcciones de 16 bits, una memoria caché de 32 entradas y un tamaño de línea de 4 bytes. ¿Cuál es el tamaño del bloque?

Como $t_{línea} = 4$ bytes, el tamaño del bloque son 4 bytes

¿Cuál es la memoria RAM máxima posible?

$$t_{RAM} = 2^{16} = 2^6 * 2^{10} = 64\text{kB}$$

¿Cuántos bloques alberga la memoria RAM suponiendo que se ha instalado el máximo posible?.

$$\text{Numero Bloques} = 2^{16} / 2^2 = 2^{14} \text{ bloques}$$

Metodos De Correspondencia

Es necesario establecer un método de correspondencia que permita saber qué bloques están cargados y en qué entrada.

Correspondencia Directa:

Cada bloque sólo puede instanciarse en una entrada predeterminada

k = número de entradas y m = número de bloques

Bloque j se instancia en la entrada $j \bmod k$

Se pueden cargar varios bloques a la misma línea

La etiqueta permite saber que bloque está cargado en la línea

- Etiqueta = Número bloque / número de entradas

Ejemplo: Tenemos una RAM de 32 kB y una caché de 32 líneas. El tamaño de la línea son 4 B. ¿En qué entrada se instancia el bloque 2004?

$j = 2004 \bmod 32 = 20 = 52 = 84 = \dots \rightarrow$ Etiqueta = $2004 / 32 = 62$

Pasos:

Dada una dirección, nos planteamos para empezar, conocer sus componentes básicos:

1. Bloque y Palabra
 - En correspondencia directa: Debido a que diferentes bloques les corresponden la misma línea de caché, debemos dividir el componente bloque en:
2. Etiqueta y línea
 - La Etiqueta permite a la caché saber que bloque está cargado

Descomposición General: Bloques, líneas y etiquetas

1. $Tam_{línea} = Tam_{bloque} \rightarrow \frac{Tam_{Memoria\ Principal}}{Tam_{bloque}} = \frac{2^p}{2^w} = 2^{(p-w)} = s \text{ bloques} \rightarrow$ Reservamos s bits para la palabra
2. $\# \text{ número líneas cache} = \frac{Tam_{cache}}{Tam_{línea}} = \frac{2^c}{2^w} = 2^{(c-w)} = r \text{ líneas} \rightarrow$ Reservamos r bits para la línea
3. $\frac{\# \text{ bloques}}{\# \text{ líneas}} = \frac{2^s}{2^r} = 2^{s-r} \rightarrow$ Reservamos $s-r$ bits para la etiqueta
Hay 2^{s-r} bloques que pueden ir a la misma línea

Ejemplo:

Caché de 64 líneas → MicroProcesador de 32 bits → Tam Bloque de 32 bytes
La CPU quiere leer la siguiente dirección #CD107908

CD107908: 1100 1101 0001 0000 0111 1001 0000 1000

¿Cuántos bits necesito para direccionar la palabra?

$32 = 2^5$ Hay 5 bits para identificar a la palabra

Palabra : 01000 (últimos 5 bits) → Valor: 8

Desplazó el número de bits para direccionar la palabra:

110 0110 1000 1000 0011 1100 1000

¿Cuántos bits son necesarios para identificar la línea?

$64 = 2^6$ → Hay 6 bits para identificar la línea

Línea: 001000 (6 bits a continuación) → Valor: 8

Desplazó el número de bits para direccionar la línea:

1 1001 1010 0010 0000 1111 → Etiqueta → #19A20F

Revisión: $\frac{2^p}{2^w} = \frac{2^{32}}{2^5} = 2^{27} \text{ bloques} \rightarrow s = 27$

$\frac{2^s}{2^r} = 2^{27-6} = 2^{21} \rightarrow \text{hay 21 bits para la etiqueta } \checkmark$

Correspondencia Asociativa:

Cualquier bloque se puede instanciar en cualquier línea del caché. La etiqueta tiene que permitir numerar todos los bloques.

Se consigue ganancia estadística frente a la correspondencia directa

Circuitaría más compleja. Circuitos que comparan etiquetas devolviendo la línea o fallo si no está instanciada..

Estos circuitos ocupan mucho en la oblea del microprocesador.

Correspondencia Asociativa por Conjuntos:

Combina las ventajas de los métodos anteriores

Los bloques se agrupan en conjuntos de tamaño fijo y se instancian siempre en las mismas líneas.

Tamaño Conjunto Cache = Tamaño Conjunto Memoria Principal

Un bloque puede instanciarse en cualquier línea del conjunto

Solo un comparador asociativo por conjunto para comparar las líneas del conjunto solo.

Líneas separadas en :

- Etiqueta
- Conjunto
- Palabra

El conjunto se mapea $C \bmod n_c$ donde C es el conjunto y n_c el número de conjuntos.

Ej 4 Hoja: Un ordenador tiene un bus de memoria de 16 bits y una caché de 1 KB. Se usa correspondencia asociativa por conjuntos con un tamaño de línea de 16 bytes y 4 líneas por conjunto. Explicar razonadamente en qué campo se divide la dirección 0000 1101 1100 0011 e indicar sus valores en decimal.

$$\text{Tam Mp} = 2^{16} \text{ bits} = 64\text{kB}$$

Tam Línea = Tam Bloque = 16 B = 2^4 B → Se reservan 4 bits para la palabra

Hay 2^2 líneas por conjunto → Se reservan 2 bits para la etiqueta

$$\text{Nº Conjuntos de Mp} = \frac{\text{Tam Mp}}{\text{Tam Conjunto}} = \frac{\text{Tam Mp}}{\text{Tam Líneas} * \text{num Líneas}} = \frac{2^{16}}{2^4 * 2^2} = 2^{10} \rightarrow \text{Se reservan 10 bits para el conjunto en memoria principal}$$

$$\text{Nº Conjuntos Mc} = \frac{\text{Tam Mc}}{\text{Tam Conjunto}} = \frac{2^{10}}{2^6} = 2^4 \rightarrow \text{Se reservan 4 bits para el conjunto en cache}$$

$$16 - (4+10) = 2 \text{ bits} \rightarrow \text{Reservados para la etiqueta}$$

Representación de la dirección: 0000 1101 1100 0011

- 4 bits para la palabra: 0011 → Valor: 3
- 10 bits para el conjunto: 00 1101 1100 → Valor: 220
- Se instancian en el conjunto 220 mod 16 en caché
- 2 bits para la etiqueta: Valor: 0

Ej 1 Hoja: Una memoria caché asociativa por conjuntos consta de 16 conjuntos con 4 particiones por conjunto. La memoria principal tiene una capacidad de 1 MB dividida en bloques de 128 palabras. a la dirección principal 1010 0001 1001 0011 0000 ¿Qué conjunto le corresponde? Exprésalo en decimal.

$$\text{Tam Mp} = 1\text{MB} = 2^{20} \text{ bytes}$$

$$\text{Num Conjuntos Cache} = 16 \rightarrow 4 \text{ líneas por conjunto}$$

$$\text{Tam Bloque} = 128 \text{ palabras} = 2^7 \text{ palabras} \rightarrow \text{Se reservan 7 bits para la palabra}$$

$$\text{Tam Mc} = \text{Tam Conjunto} * \text{Num Conjuntos} = 2^4 * 2^2 = 2^6 \text{ líneas} = 2^6 * 2^7 = 2^{13} \text{ bytes} = 8\text{kB}$$

$$\text{Num Conjuntos Mp} = \frac{\text{Tam Mp}}{\text{Tam Conjunto}} = \frac{2^{20}}{2^7 * 2^2} = 2^{11} \rightarrow \text{Se reservan 11 bits para el conjunto}$$

$$20 - (11 + 7) = 2 \text{ bits para la etiqueta}$$

$$\text{Palabra (7 bits): } 011 \text{ } 0000 \rightarrow \text{Valor: } 48$$

$$\text{Conjunto (11 bits): } 10 \text{ } 0001 \text{ } 1001 \text{ } 0 \rightarrow \text{Valor: } 1074 \rightarrow \text{En conjunto : } 1074 \text{ mod } 128$$

$$\text{Etiqueta (2 bits): } 10 \rightarrow \text{Valor: } 2$$

Ej 6: Sean direcciones de memoria de 10 bits. Tenemos además una caché con correspondencia asociativa por conjuntos, donde cada conjunto tiene 2 líneas. En cada línea de caché cabe un bloque de 8 palabras. ¿En cuántos bloques está dividida

la memoria principal? ¿Cuál es el tamaño que debe tener la caché (en conjunto, líneas y palabras) para que a cada conjunto de correspondan 8 bloques de memoria principal?

Tam Conjunto = Tam Bloque * N° líneas $\rightarrow$ Tam Conj = $2^3 * 2 = 2^4 \rightarrow$ Se reservan 4 bits para el conjunto

Tam Mp = 2^{10} bytes = 1kB

N° Bloques Mp = $\frac{Tam\ Mp}{Tam\ Bloque} = \frac{2^{10}}{2^3} = 2^7 \rightarrow$ Hay 128 bloques en Mp

N° Conjuntos Mp = $\frac{2^7}{2} = 2^6 = 64$ conjuntos $\rightarrow$ 6 bits por conjunto

Tam Conjunto Cache = $64/8 = 8 = 2^3$ bytes

Hay 16 líneas en cache

Cache En Varios Niveles

Hay varios niveles de caché:

Nivel 1: El más pequeño y rápido

Cada caché es el cache del nivel previo

La comunicación en cache es mas rapida que en RAM

Políticas de Reemplazo

Cuando se produce un fallo, la caché debe saber que línea expulsar para reemplazarla.

Si la correspondencia es directa:

- Política Trivial. Se debe reemplazar la línea afectada

Si la correspondencia es asociativa:

- Reemplazo Aleatorio: Se decide un línea al azar
- FIFO: Se reemplaza la línea que lleve más tiempo cargada
- LRU: Se expulsa la línea que lleva más tiempo sin ser referenciada
- LFU: Se expulsa la línea que se haya referenciado menos veces

Políticas de Escritura

Cada vez que se escribe un dato en la caché, la RAM se queda desactualizada. Hay que sincronizarla.

- **Write Through:** Cada vez que hay escritura en caché, se reescribe el bloque el la RAM.
- **Write Back:** La línea solo se escribe en RAM si está sucia en el momento de ser reemplazada